

Exercise 4 - Sample distribution and Central Limit Theorem

Florescu Andrei - 12242987

2024

Task 1

```
set.seed(12242987)
data_points <- c(4.94, 5.06, 4.53, 5.07, 4.99, 5.16, 4.38, 4.43, 4.93, 4.72, 4.92, 4.96) # nolint
```

1. Number of possible bootstrap samples

Since every position can take value as any of the data points and the number of available positions is 12, therefore:

$$Total_bootstrap_samples = 12^{12}$$

```
12**12
```

```
## [1] 8.9161e+12
```

2. Mean and the median of the original sample

Mean:

```
mean_data <- mean(data_points)
mean_data
```

```
## [1] 4.840833
```

Median:

```
median_data <- median(data_points)
median_data
```

```
## [1] 4.935
```

3. Create 2000 bootstrap samples and compute their means

```
num_samples <- 2000
n <- length(data_points)
bootstrap_means <- replicate(
  num_samples,
  mean(sample(data_points, n, replace = TRUE))
)

head(bootstrap_means)
```

```
## [1] 4.879167 4.875833 4.957500 4.795833 4.870000 4.753333
```

1. Compute the mean on the first 20 bootstrap means

```
mean_20 <- mean(bootstrap_means[1:20])
mean_20
```

```
## [1] 4.84875
```

2. Compute the mean of the first 200 bootstrap means

```
mean_200 <- mean(bootstrap_means[1:200])
mean_200
```

```
## [1] 4.840704
```

3. Compute the mean based on all 2000 bootstrap means

```
mean_2000 <- mean(bootstrap_means)
mean_2000
```

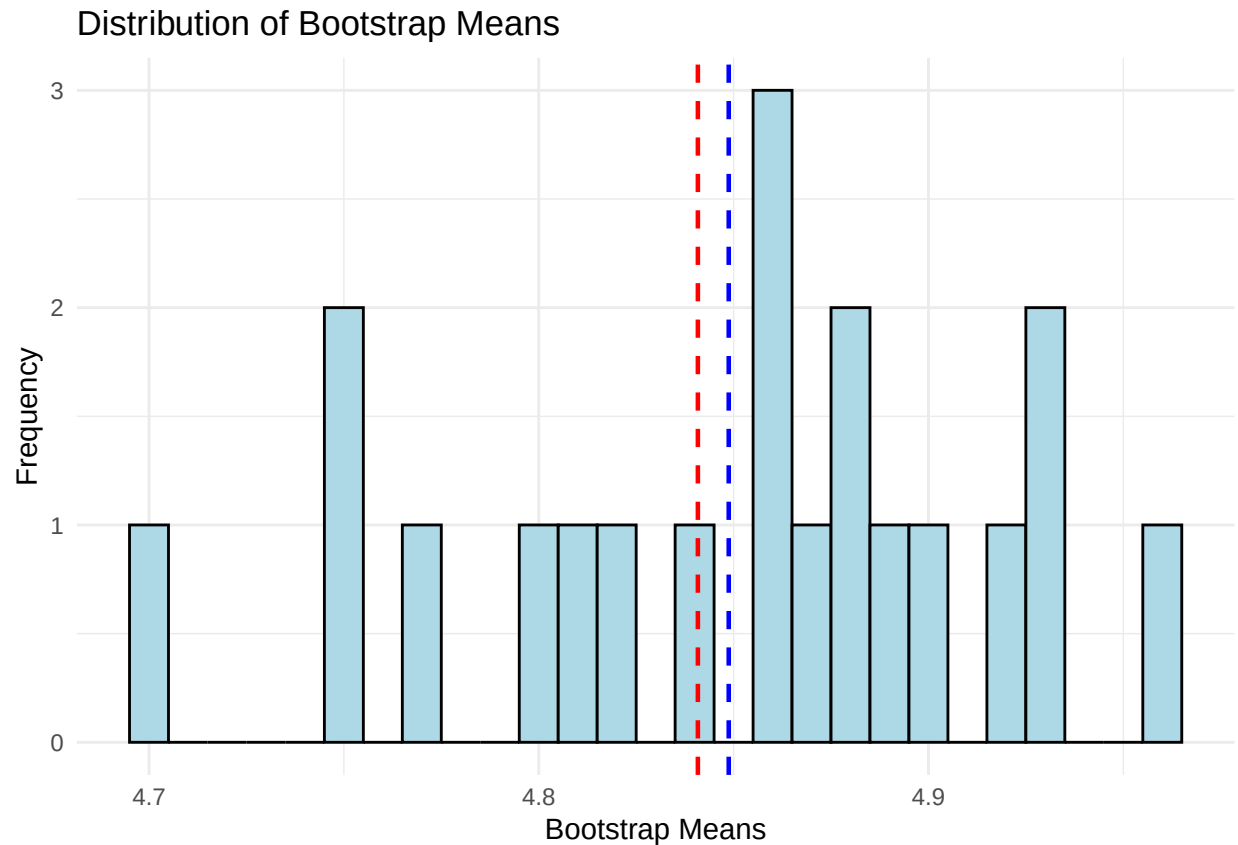
```
## [1] 4.842377
```

4. Visualise the distribution all the different bootstrap means

```
library(ggplot2)
```

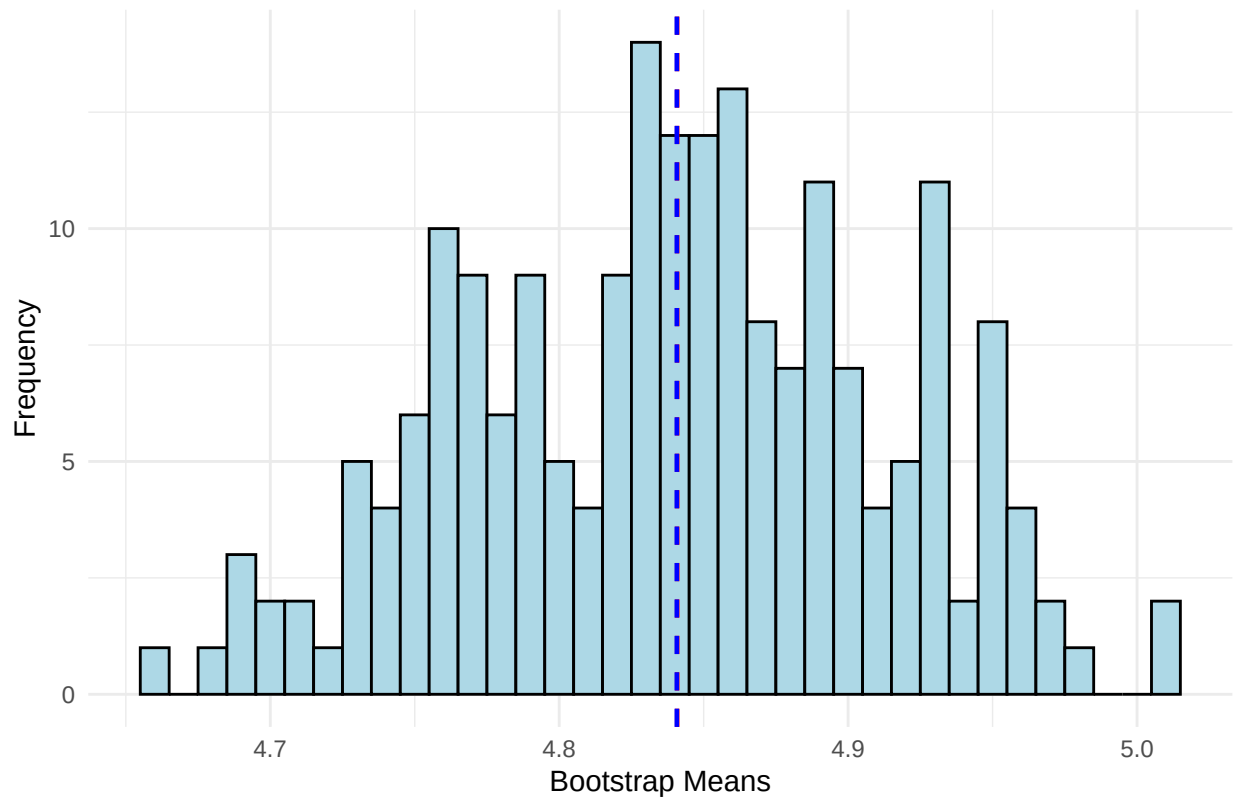
```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
ggplot(
  data.frame(bootstrap_means[1:20]),
  aes(x = bootstrap_means[1:20])
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = mean_data,
    color = "red",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  geom_vline(
    xintercept = mean_20,
    color = "blue",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  labs(
    title = "Distribution of Bootstrap Means",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```



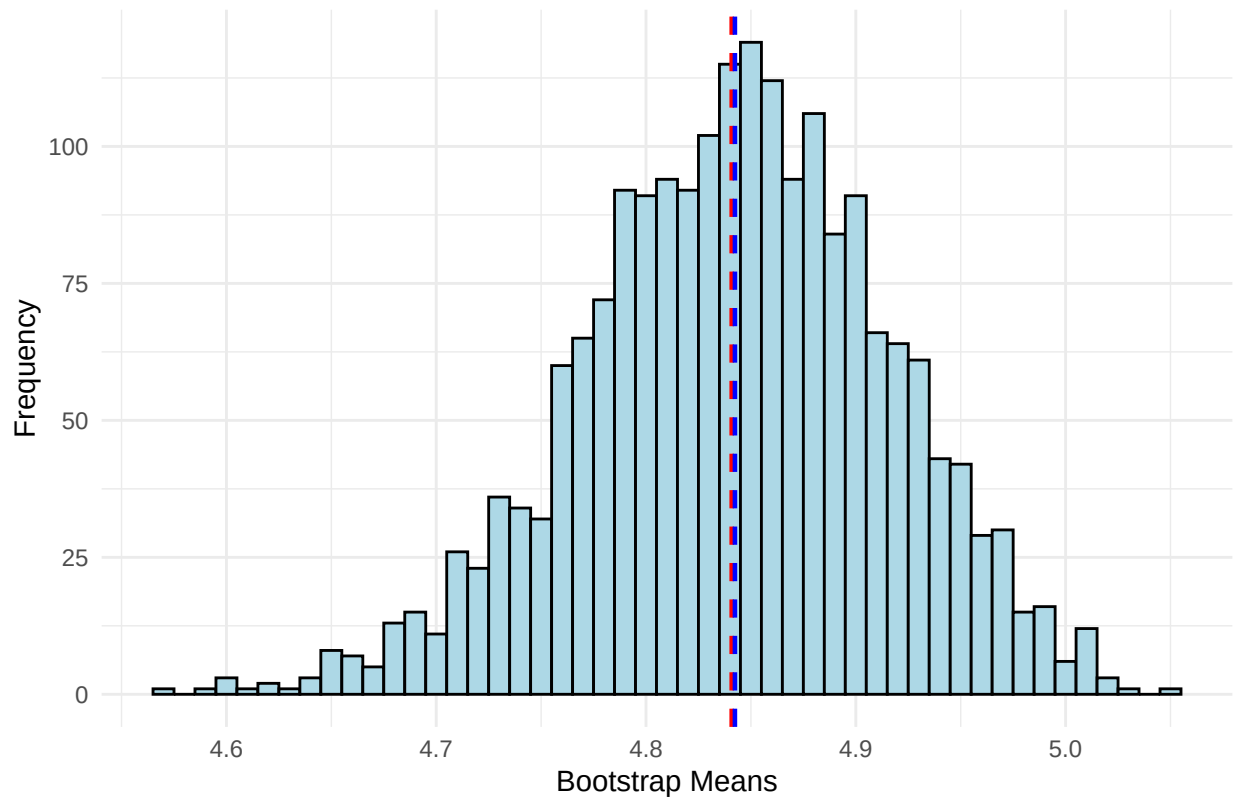
```
ggplot(
  data.frame(bootstrap_means[1:200]),
  aes(x = bootstrap_means[1:200])
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = mean_data,
    color = "red",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  geom_vline(
    xintercept = mean_200,
    color = "blue",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  labs(
    title = "Distribution of Bootstrap Means",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```

Distribution of Bootstrap Means



```
ggplot(
  data.frame(bootstrap_means),
  aes(x = bootstrap_means)
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = mean_data,
    color = "red",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  geom_vline(
    xintercept = mean_2000,
    color = "blue",
    linetype = "dashed",
    linewidth = 0.8
  ) +
  labs(
    title = "Distribution of Bootstrap Means",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```

Distribution of Bootstrap Means



We can see that the central limit theorem kicks in because if we increase the number of samples, the distribution is closer to a normal distribution.

5. Based on the three different bootstrap sample lengths compute the corresponding 0.025 and 0.975 quantiles

```
quantiles_20 <- quantile(bootstrap_means[1:20], probs = c(0.025, 0.975))
quantiles_200 <- quantile(bootstrap_means[1:200], probs = c(0.025, 0.975))
quantiles_2000 <- quantile(bootstrap_means, probs = c(0.025, 0.975))
```

```
quantiles_20
```

```
##      2.5%      97.5%
## 4.727521 4.944437
```

```
quantiles_200
```

```
##      2.5%      97.5%
## 4.700687 4.962688
```

```
quantiles_2000
```

```
##      2.5%      97.5%
## 4.690812 4.977500
```

```
t_test <- t.test(data_points, conf.level = 0.95)
```

```
true_ci <- t_test$conf.int
```

```

true_ci

## [1] 4.674344 5.007323
## attr(,"conf.level")
## [1] 0.95

list(
  Quantiles_20 = quantiles_20,
  Quantiles_200 = quantiles_200,
  Quantiles_2000 = quantiles_2000,
  True_CI = true_ci
)

## $Quantiles_20
##      2.5%      97.5%
## 4.727521 4.944437
##
## $Quantiles_200
##      2.5%      97.5%
## 4.700687 4.962688
##
## $Quantiles_2000
##      2.5%      97.5%
## 4.690812 4.977500
##
## $True_CI
## [1] 4.674344 5.007323
## attr(,"conf.level")
## [1] 0.95

```

4. Create 2000 bootstrap samples and compute their medians

```

num_samples <- 2000
n <- length(data_points)
bootstrap_medians <- replicate(
  num_samples,
  median(sample(data_points, n, replace = TRUE))
)

head(bootstrap_medians)

```

```
## [1] 4.925 4.940 4.930 4.930 4.950 4.960
```

1. Compute the median on the first 20 bootstrap medians

```

median_20 <- median(bootstrap_medians[1:20])
median_20

## [1] 4.94

```

2. Compute the median of the first 200 bootstrap medians

```

median_200 <- median(bootstrap_medians[1:200])
median_200

```

```
## [1] 4.935
```

3. Compute the median based on all 2000 bootstrap medians

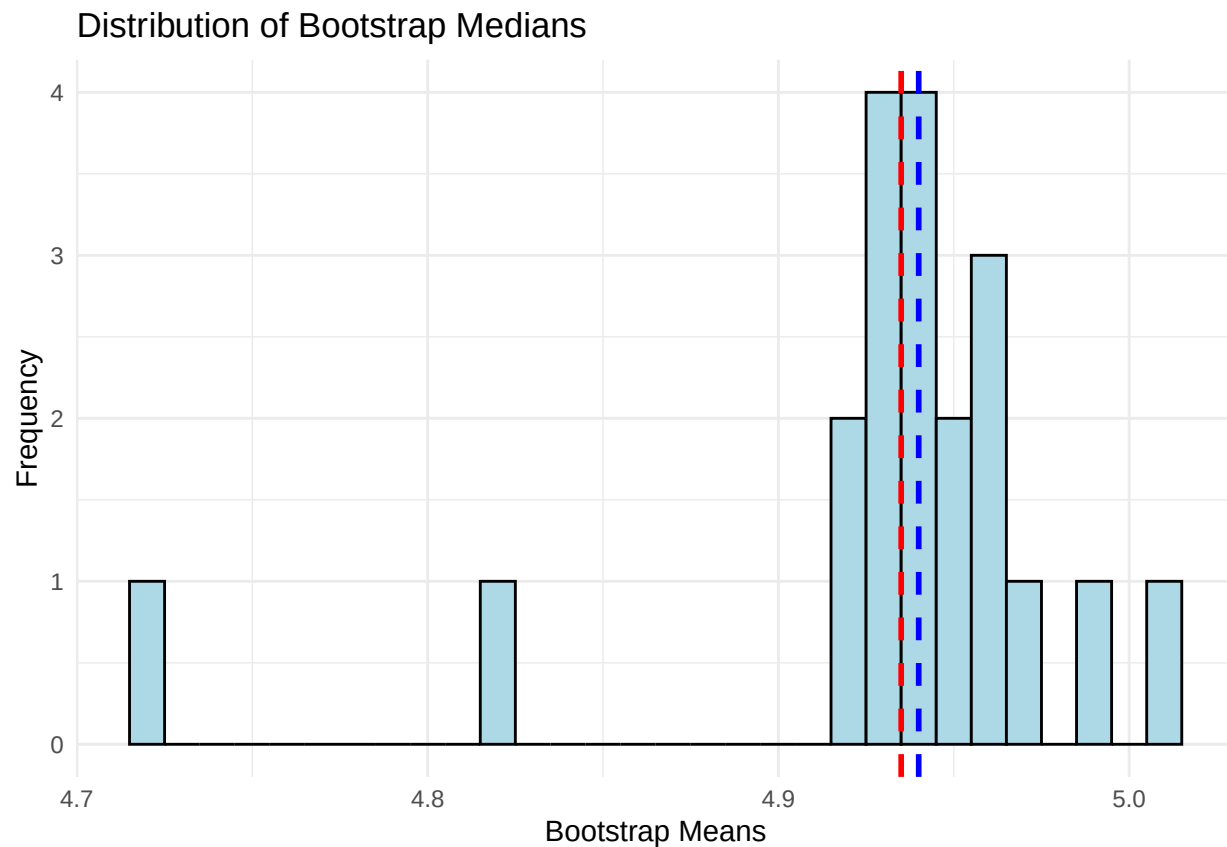
```
median_2000 <- median(bootstrap_medians)
median_2000
```

```
## [1] 4.935
```

4. Visualise the distribution all the different bootstrap medians

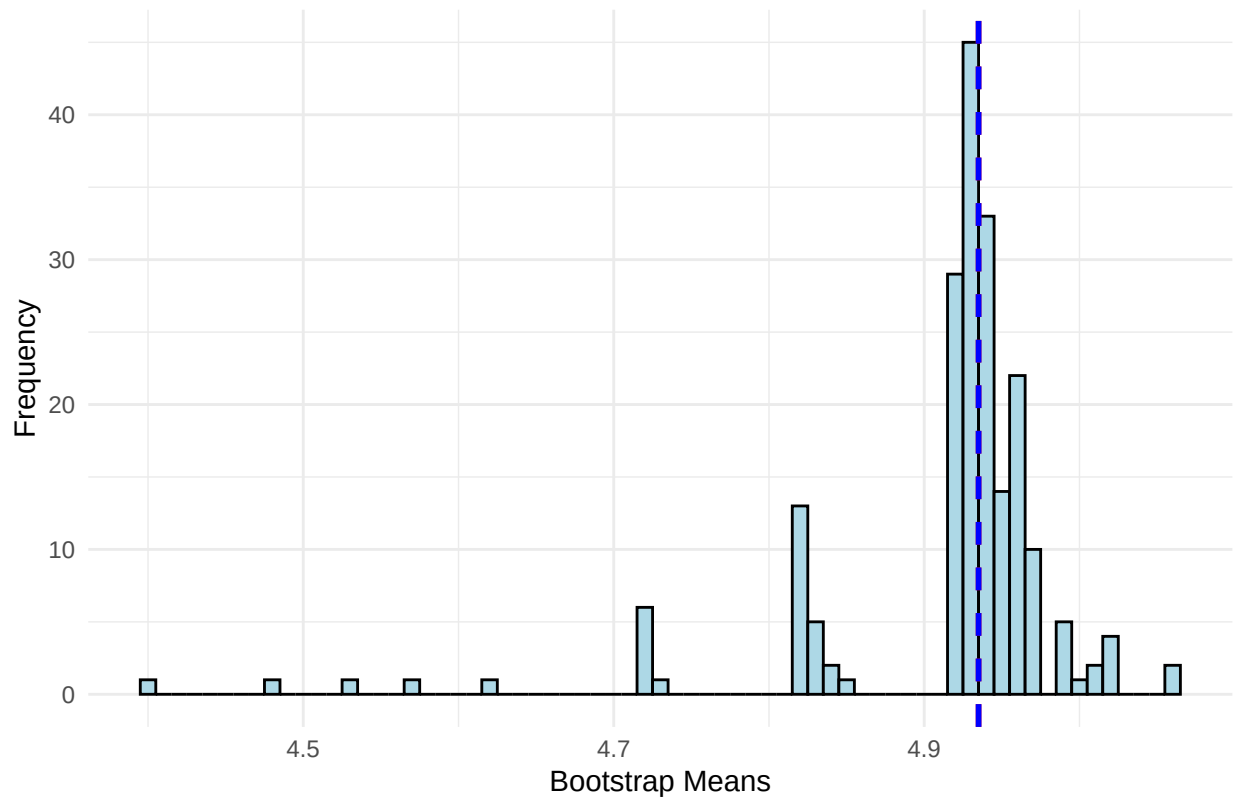
```
library(ggplot2)

ggplot(
  data.frame(bootstrap_medians[1:20]),
  aes(x = bootstrap_medians[1:20])
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = median_data,
    color = "red",
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = median_20,
    color = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution of Bootstrap Medians",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```



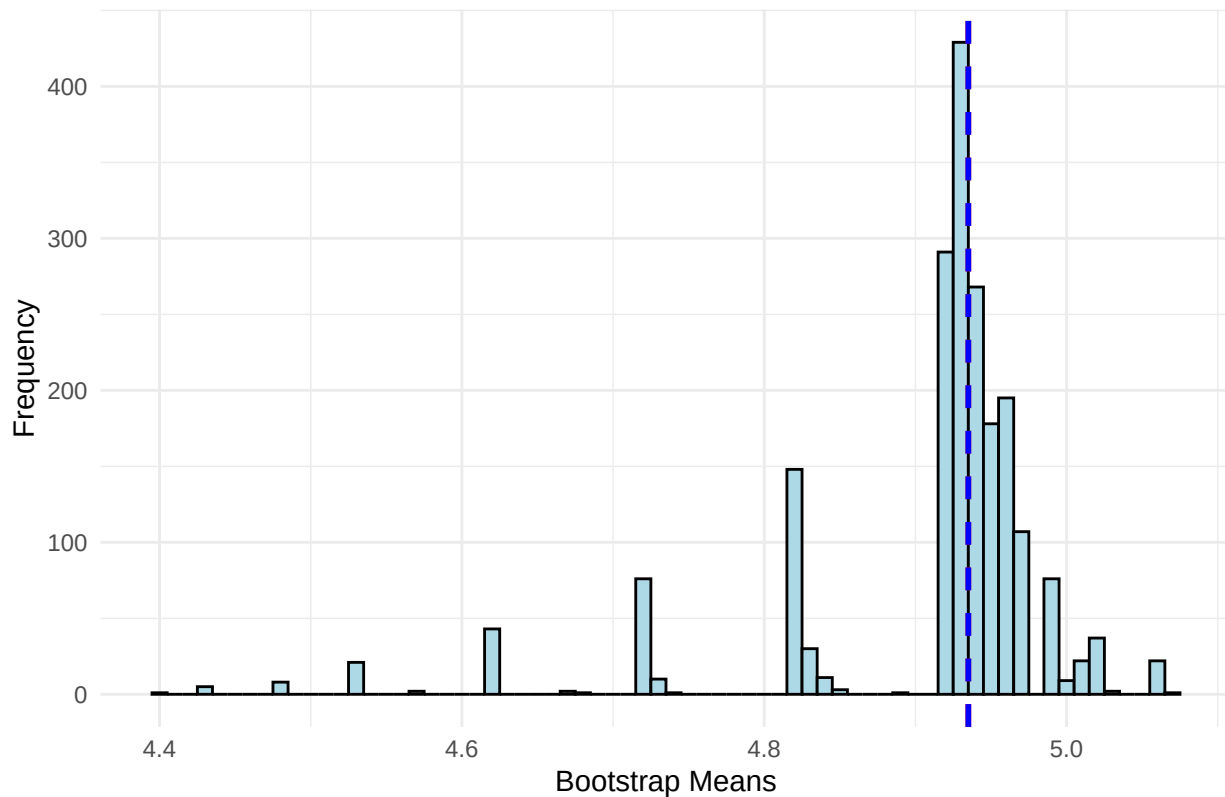
```
ggplot(
  data.frame(bootstrap_medians[1:200]),
  aes(x = bootstrap_medians[1:200])
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = median_data,
    color = "red",
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = median_200,
    color = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution of Bootstrap Medians",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```


Distribution of Bootstrap Medians



```
ggplot(
  data.frame(bootstrap_medians),
  aes(x = bootstrap_medians)
) +
  geom_histogram(binwidth = 0.01, color = "black", fill = "lightblue") +
  geom_vline(
    xintercept = median_data,
    color = "red",
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = median_2000,
    color = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution of Bootstrap Medians",
    x = "Bootstrap Means",
    y = "Frequency"
  ) +
  theme_minimal()
```

Distribution of Bootstrap Medians



5. Based on the three different bootstrap sample lengths compute the corresponding 0.025 and 0.975 quantiles

```
quantiles_20 <- quantile(bootstrap_medians[1:20], probs = c(0.025, 0.975))
quantiles_200 <- quantile(bootstrap_medians[1:200], probs = c(0.025, 0.975))
quantiles_2000 <- quantile(bootstrap_medians, probs = c(0.025, 0.975))
```

```
quantiles_20
```

```
## 2.5% 97.5%
## 4.7725 5.0005
```

```
quantiles_200
```

```
## 2.5% 97.5%
## 4.717625 5.025000
```

```
quantiles_2000
```

```
## 2.5% 97.5%
## 4.625 5.025
```

```
t_test <- t.test(data_points, conf.level = 0.95)
```

```
true_ci <- t_test$conf.int
```

```
true_ci
```

```
## [1] 4.674344 5.007323
## attr(,"conf.level")
## [1] 0.95

list(
  Quantiles_20 = quantiles_20,
  Quantiles_200 = quantiles_200,
  Quantiles_2000 = quantiles_2000,
  True_CI = true_ci
)

## $Quantiles_20
##   2.5% 97.5%
## 4.7725 5.0005
##
## $Quantiles_200
##   2.5% 97.5%
## 4.717625 5.025000
##
## $Quantiles_2000
##   2.5% 97.5%
## 4.625 5.025
##
## $True_CI
## [1] 4.674344 5.007323
## attr(,"conf.level")
## [1] 0.95
```

Task 2

1. Generate x.clean, x.cont and x

```
set.seed(1234)
x.clean <- rnorm(1960, mean = 0, sd = 1)
x.cont <- runif(40, min = 4, max = 5)
x <- c(x.clean, x.cont)
set.seed(12242987)
head(x)

## [1] -1.2070657 0.2774292 1.0844412 -2.3456977 0.4291247 0.5060559
```

2. Estimate the median, the mean and the trimmed mean with $\alpha = 0.05$ for x and x.clean.

Estimations for x:

```
median_x <- median(x)
mean_x <- mean(x)
trimmed_mean_x <- mean(x, trim = 0.05)
```

Estimations for x.clean:

```
median_x_clean <- median(x.clean)
mean_x_clean <- mean(x.clean)
trimmed_mean_x_clean <- mean(x.clean, trim = 0.05)
```

3. Use nonparametric bootstrap (for x and x.clean)

```
non_param_bootstrap <- function(data, stat_function, N = 2000, trim = NULL) {
  n <- length(data)
  boot_stats <- numeric(N)

  for (i in 1:N) {
    sample_data <- sample(data, size = n, replace = TRUE)
    if (!is.null(trim)) {
      boot_stats[i] <- stat_function(sample_data, trim = trim)
    } else {
      boot_stats[i] <- stat_function(sample_data)
    }
  }

  std_err <- sd(boot_stats)

  conf_int <- quantile(boot_stats, probs = c(0.025, 0.975))

  list(boot_stats = boot_stats, se = std_err, ci = conf_int)
}

datasets <- list(x = x, x_clean = x.clean)

results <- lapply(datasets, function(data) {
  list(
    median = non_param_bootstrap(data, median),
    mean = non_param_bootstrap(data, mean),
    trimmed_mean = non_param_bootstrap(data, mean, trim = 0.05)
  )
})

result_summary <- lapply(results, function(estimates) {
  sapply(estimates, function(res) {
    c(SE = res$se, CI_Lower = res$ci[1], CI_Upper = res$ci[2])
  })
})

result_summary
```



```
## $x
##           median           mean trimmed_mean
## SE           0.02889251 0.02565898 0.022814029
## CI_Lower.2.5% -0.04922111 0.03465418 -0.006588443
## CI_Upper.97.5% 0.06522648 0.13494750 0.083782837
##
## $x_clean
##           median           mean trimmed_mean
## SE           0.02720911 0.02248115 0.02288631
## CI_Lower.2.5% -0.06777026 -0.05016273 -0.04797475
## CI_Upper.97.5% 0.03809420 0.03780455 0.04387607
```

Here i created a function that returns the stats requested for a bootstrapped dataset. As parameters the function receives the data used, the estimation function to be used and the number of samples (trim is optional for a trimmed mean). Then i applied this function on each dataset and for each estimation function and displayed the results

4.

```
parametric_bootstrap <- function(data, stat_function, B = 2000, trim = NULL, robust = FALSE) {
  n <- length(data)

  mu_hat <- mean(data)
  if (robust) {
    sigma <- mad(data) # use mad for robust scale estimation
  } else {
    sigma <- sd(data)
  }

  boot_stats <- numeric(B)
  for (i in 1:B) {
    boot_sample <- rnorm(n, mean = mu_hat, sd = sigma) # sample randomly from normal dist
    if (!is.null(trim)) {
      boot_stats[i] <- stat_function(boot_sample, trim = trim)
    } else {
      boot_stats[i] <- stat_function(boot_sample)
    }
  }

  obs_stat <- if (!is.null(trim)) {
    stat_function(data, trim = trim)
  } else {
    stat_function(data)
  }

  # calculate bootstrap bias, standard error and percentile confidence interval
  bias <- mean(boot_stats) - obs_stat
  se <- sd(boot_stats)
  ci <- quantile(boot_stats, probs = c(0.025, 0.975))
  bias_corrected <- obs_stat - bias

  list(
    Observed = obs_stat,
    Bias = bias,
    SE = se,
    CI = ci,
    Bias_Corrected = bias_corrected
  )
}

results <- list(
  x = list(
    Mean = parametric_bootstrap(x, mean, robust = FALSE),
    Trimmed_Mean = parametric_bootstrap(x, mean, trim = 0.05, robust = FALSE),
    Mean_Robust = parametric_bootstrap(x, mean, robust = TRUE),
    Trimmed_Mean_Robust = parametric_bootstrap(x, mean, trim = 0.05, robust = TRUE)
  ),
  x_clean = list(
    Mean = parametric_bootstrap(x_clean, mean, robust = FALSE),
    Trimmed_Mean = parametric_bootstrap(x_clean, mean, trim = 0.05, robust = FALSE),
    Mean_Robust = parametric_bootstrap(x_clean, mean, robust = TRUE),
  )
)
```

```

    Trimmed_Mean_Robust = parametric_bootstrap(x.clean, mean, trim = 0.05, robust = TRUE)
  )
)
results

## $x
## $x$Mean
## $x$Mean$Observed
## [1] 0.08395508
##
## $x$Mean$Bias
## [1] 3.099887e-05
##
## $x$Mean$SE
## [1] 0.02598709
##
## $x$Mean$CI
##      2.5%      97.5%
## 0.03193625 0.13537902
##
## $x$Mean$Bias_Corrected
## [1] 0.08392408
##
##
## $x$Trimmed_Mean
## $x$Trimmed_Mean$Observed
## [1] 0.03683294
##
## $x$Trimmed_Mean$Bias
## [1] 0.04644734
##
## $x$Trimmed_Mean$SE
## [1] 0.02710043
##
## $x$Trimmed_Mean$CI
##      2.5%      97.5%
## 0.03098484 0.13643062
##
## $x$Trimmed_Mean$Bias_Corrected
## [1] -0.009614404
##
##
## $x$Mean_Robust
## $x$Mean_Robust$Observed
## [1] 0.08395508
##
## $x$Mean_Robust$Bias
## [1] -0.0006712824
##
## $x$Mean_Robust$SE
## [1] 0.02214248
##
## $x$Mean_Robust$CI
##      2.5%      97.5%

```

```

## 0.04036133 0.12679824
##
## $x$Mean_Robust$Bias_Corrected
## [1] 0.08462636
##
##
## $x$Trimmed_Mean_Robust
## $x$Trimmed_Mean_Robust$Observed
## [1] 0.03683294
##
## $x$Trimmed_Mean_Robust$Bias
## [1] 0.04722455
##
## $x$Trimmed_Mean_Robust$SE
## [1] 0.02259696
##
## $x$Trimmed_Mean_Robust$CI
##      2.5%      97.5%
## 0.03946337 0.12647234
##
## $x$Trimmed_Mean_Robust$Bias_Corrected
## [1] -0.01039161
##
##
##
## $x_clean
## $x_clean$Mean
## $x_clean$Mean$Observed
## [1] -0.005968976
##
## $x_clean$Mean$Bias
## [1] -0.0008670181
##
## $x_clean$Mean$SE
## [1] 0.02325919
##
## $x_clean$Mean$CI
##      2.5%      97.5%
## -0.05064846 0.03961283
##
## $x_clean$Mean$Bias_Corrected
## [1] -0.005101958
##
##
## $x_clean$Trimmed_Mean
## $x_clean$Trimmed_Mean$Observed
## [1] -0.001462623
##
## $x_clean$Trimmed_Mean$Bias
## [1] -0.004520157
##
## $x_clean$Trimmed_Mean$SE
## [1] 0.02289341
##

```

```
## $x_clean$Trimmed_Mean$CI
##      2.5%      97.5%
## -0.04858496  0.04006762
##
## $x_clean$Trimmed_Mean$Bias_Corrected
## [1] 0.003057533
##
##
## $x_clean$Mean_Robust
## $x_clean$Mean_Robust$Observed
## [1] -0.005968976
##
## $x_clean$Mean_Robust$Bias
## [1] 0.00054646
##
## $x_clean$Mean_Robust$SE
## [1] 0.02173096
##
## $x_clean$Mean_Robust$CI
##      2.5%      97.5%
## -0.04767361  0.03674108
##
## $x_clean$Mean_Robust$Bias_Corrected
## [1] -0.006515436
##
##
## $x_clean$Trimmed_Mean_Robust
## $x_clean$Trimmed_Mean_Robust$Observed
## [1] -0.001462623
##
## $x_clean$Trimmed_Mean_Robust$Bias
## [1] -0.004130005
##
## $x_clean$Trimmed_Mean_Robust$SE
## [1] 0.02237077
##
## $x_clean$Trimmed_Mean_Robust$CI
##      2.5%      97.5%
## -0.04827495  0.03861455
##
## $x_clean$Trimmed_Mean_Robust$Bias_Corrected
## [1] 0.002667382
```

5.

```
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.4.2
```

```
# combine results for 3. into a single data frame
```

```
result_df <- do.call(rbind, lapply(names(results), function(dataset) {
  do.call(rbind, lapply(names(results[[dataset]]), function(stat) {
    res <- results[[dataset]][[stat]]
    data.frame(
```



```

    Dataset = dataset,
    Statistic = stat,
    Observed = res$Observed,
    Bias = res$Bias,
    Standard_Error = res$SE,
    CI_Lower = res$CI[1],
    CI_Upper = res$CI[2],
    Bias_Corrected = res$Bias_Corrected
  )
}))
}))

kable(result_df, caption = "Non Parametric Bootstrap Results Summary")

```

Table 1: Non Parametric Bootstrap Results Summary

	Dataset	Statistic	Observed	Bias	Standard_Error	CI_Lower	CI_Upper	Bias_Corrected
2.5%	x	Mean	0.0839551	0.0000310	0.0259871	0.0319362	0.1353790	0.0839241
2.5%1	x	Trimmed_Mean	0.0368329	0.0464473	0.0271004	0.0309848	0.1364306	-0.0096144
2.5%2	x	Mean_Robust	0.0839551	-	0.0221425	0.0403613	0.1267982	0.0846264
				0.0006713				
2.5%3	x	Trimmed_Mean_Robust	0.0368329	0.0472245	0.0225970	0.0394634	0.1264723	-0.0103916
2.5%4	x_clean	Mean	-	-	0.0232592	-	0.0396128	-0.0051020
			0.0059690	0.0008670		0.0506485		
2.5%11	x_clean	Trimmed_Mean	-	-	0.0228934	-	0.0400676	0.0030575
			0.0014626	0.0045202		0.0485850		
2.5%21	x_clean	Mean_Robust	-	0.0005465	0.0217310	-	0.0367411	-0.0065154
			0.0059690			0.0476736		
2.5%31	x_clean	Trimmed_Mean_Robust	-	-	0.0223708	-	0.0386146	0.0026674
			0.0014626	0.0041300		0.0482750		

```

format_results <- function(res, dataset_name, stat_name) {
  data.frame(
    Dataset = dataset_name,
    Statistic = stat_name,
    Observed = res$Observed,
    Bias = res$Bias,
    Standard_Error = res$SE,
    CI_Lower = res$CI[1],
    CI_Upper = res$CI[2],
    Bias_Corrected = res$Bias_Corrected
  )
}

# combine results for 4. into a single data frame
result_df <- do.call(rbind, lapply(names(results), function(dataset) {
  do.call(rbind, lapply(names(results)[[dataset]]), function(stat) {
    format_results(results[[dataset]][[stat]], dataset, stat)
  })
}))
}))

kable(result_df, caption = "Parametric Bootstrap Results Summary")

```

Table 2: Parametric Bootstrap Results Summary

	Dataset	Statistic	Observed	Bias	Standard_Error	CI_Lower	CI_Upper	Bias_Corrected
2.5%	x	Mean	0.0839551	0.0000310	0.0259871	0.0319362	0.1353790	0.0839241
2.5%1	x	Trimmed_Mean	0.0368329	0.0464473	0.0271004	0.0309848	0.1364306	-0.0096144
2.5%2	x	Mean_Robust	0.0839551	-	0.0221425	0.0403613	0.1267982	0.0846264
				0.0006713				
2.5%3	x	Trimmed_Mean_Robust	0.0368329	0.0472245	0.0225970	0.0394634	0.1264723	-0.0103916
2.5%4	x_clean	Mean	-	-	0.0232592	-	0.0396128	-0.0051020
			0.0059690	0.0008670		0.0506485		
2.5%11	x_clean	Trimmed_Mean	-	-	0.0228934	-	0.0400676	0.0030575
			0.0014626	0.0045202		0.0485850		
2.5%21	x_clean	Mean_Robust	-	0.0005465	0.0217310	-	0.0367411	-0.0065154
			0.0059690			0.0476736		
2.5%31	x_clean	Trimmed_Mean_Robust	-	-	0.0223708	-	0.0386146	0.0026674
			0.0014626	0.0041300		0.0482750		

Task 3

Bootstrapping is a great tool to resample data by using the given data to estimate the sampling distribution. For non parametric bootstrapping the methodology can be described as follows:

1. We chose a statistic function to use (in our cases we used the mean, median and trimmed mean)
2. We resampled n data points from the original dataset and we generated it 2000 times
3. We computed the statistics for each bootstrap sample resulting in bootstrap estimates
4. Then we computed the confidence intervals and the standard deviation

For the parametric bootstrapping the methodology is described as follows:

1. We chose a statistic function to use (this time we used only mean and trimmed mean)
2. We assumed the data follows a statistic distribution (the normal distribution in our case)
3. As a standard deviation we used either the classical standard deviation function or the MAD (if robust)
4. We resampled n data points from the original dataset 2000 times
5. We computed the statistics for each sample resulting in the bootstrap estimates
6. Then we computed the bias, standard error, 95 percentile CI and bias corrected estimate